

## Simple Analogy

| Concept         | Analogy                                                         |
|-----------------|-----------------------------------------------------------------|
| @RequestMapping | Like setting the <b>address</b> where mail is delivered (route) |
| @RequestBody    | Like the <b>content</b> of the mail (data sent in request body) |

Would you like me to also explain how @RequestMapping **differs from** @GetMapping , @PostMapping , **etc.** (these are often follow-up questions in interviews)?



any annoattaion remaining ?

Excellent — you're thinking like an interviewer now 🔥

Let's go through **all the important and frequently used Spring MVC + Spring Boot annotations** — categorized for easy recall during interviews.

## 1 Core Spring MVC Annotations

| Annotation      | Purpose                                                                                 | Example                                       |
|-----------------|-----------------------------------------------------------------------------------------|-----------------------------------------------|
| @Controller     | Marks a class as a <b>Spring MVC controller</b> (returns view pages like JSP/Thymeleaf) | @Controller public class HomeController {}    |
| @RestController | Combines @Controller + @ResponseBody → returns JSON instead of a view                   | @RestController public class ApiController {} |

| Annotation                                                                                                                                          | Purpose                                                                                   | Example                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>@RequestMapping</code>                                                                                                                        | Maps HTTP requests to controller classes or methods                                       | <code>@RequestMapping("/users")</code>                       |
| <code>@GetMapping</code> , <code>@PostMapping</code> ,<br><code>@PutMapping</code> ,<br><code>@DeleteMapping</code> ,<br><code>@PatchMapping</code> | Shortcut annotations for HTTP verbs (replaces <code>@RequestMapping(method=... )</code> ) | <code>@GetMapping("/users/{id}")</code>                      |
| <code>@RequestParam</code>                                                                                                                          | Binds query parameters ( <code>? id=5</code> ) to method parameters                       | <code>@RequestParam("id") int id</code>                      |
| <code>@PathVariable</code>                                                                                                                          | Binds URI path variables ( <code>/users/5</code> ) to method parameters                   | <code>@PathVariable("id") int id</code>                      |
| <code>@RequestBody</code>                                                                                                                           | Binds request body JSON/XML to a Java object                                              | <code>@RequestBody User user</code>                          |
| <code>@ResponseBody</code>                                                                                                                          | Sends method return value as the response body (usually JSON)                             | <code>@ResponseBody public String getData()</code>           |
| <code>@ModelAttribute</code>                                                                                                                        | Binds form data (from HTML forms) to a model object                                       | <code>@ModelAttribute User user</code>                       |
| <code>@CrossOrigin</code>                                                                                                                           | Enables CORS for a controller or method                                                   | <code>@CrossOrigin(origins = "http://localhost:4200")</code> |



2

## Dependency Injection & Bean Management

| Annotation              | Purpose                                                     | Example                                              |
|-------------------------|-------------------------------------------------------------|------------------------------------------------------|
| <code>@Component</code> | Marks a class as a Spring-managed bean                      | <code>@Component public class EmailService {}</code> |
| <code>@Service</code>   | Specialization of <code>@Component</code> for service layer | <code>@Service public class UserService {}</code>    |

| Annotation                  | Purpose                                                                                       | Example                                                                                                                                  |
|-----------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@Repository</code>    | Specialization of <code>@Component</code> for DAO layer (enables exception translation)       | <code>@Repository</code> public interface<br><code>UserRepository {}</code>                                                              |
| <code>@Autowired</code>     | Injects dependencies automatically                                                            | <code>@Autowired</code> private <code>UserService</code><br><code>userService;</code>                                                    |
| <code>@Qualifier</code>     | Used with <code>@Autowired</code> to specify which bean to inject when multiple are available | <code>@Qualifier("emailService")</code>                                                                                                  |
| <code>@Value</code>         | Injects values from properties or environment variables                                       | <code>@Value("\${app.name}")</code> private<br><code>String appName;</code>                                                              |
| <code>@Configuration</code> | Marks a class as a source of bean definitions                                                 | <code>@Configuration</code> public class<br><code>AppConfig {}</code>                                                                    |
| <code>@Bean</code>          | Defines a bean explicitly within a <code>@Configuration</code> class                          | <code>@Bean</code> public <code>PasswordEncoder</code><br><code>encoder() { return new</code><br><code>BCryptPasswordEncoder(); }</code> |



3

## Spring Security Annotations

| Annotation                         | Purpose                                                            | Example                                                                        |
|------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>@EnableWebSecurity</code>    | Enables Spring Security configuration                              | <code>@EnableWebSecurity</code> public class<br><code>SecurityConfig {}</code> |
| <code>@EnableMethodSecurity</code> | Enables method-level security ( <code>@PreAuthorize</code> , etc.) | <code>@EnableMethodSecurity</code>                                             |
| <code>@PreAuthorize</code>         | Runs before method execution (uses SpEL)                           | <code>@PreAuthorize("hasRole('ADMIN')")</code><br><code>)</code>               |
| <code>@PostAuthorize</code>        | Runs after method execution                                        | <code>@PostAuthorize("returnObject.owner == authentication.name")</code>       |
| <code>@Secured</code>              | Restricts access by roles                                          | <code>@Secured("ROLE_ADMIN")</code>                                            |

| Annotation                 | Purpose                                        | Example                                                       |
|----------------------------|------------------------------------------------|---------------------------------------------------------------|
| <code>@WithMockUser</code> | Used in unit tests to simulate logged-in users | <code>@WithMockUser(username="admin", roles={"ADMIN"})</code> |

## JPA & Database Annotations

| Annotation                                                                                            | Purpose                                            | Example                                                          |
|-------------------------------------------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------|
| <code>@Entity</code>                                                                                  | Marks a class as a JPA entity (mapped to DB table) | <code>@Entity public class User {}</code>                        |
| <code>@Table</code>                                                                                   | Specifies table name for an entity                 | <code>@Table(name = "users")</code>                              |
| <code>@Id</code>                                                                                      | Marks primary key field                            | <code>@Id private Long id;</code>                                |
| <code>@GeneratedValue</code>                                                                          | Auto-generates primary key values                  | <code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code> |
| <code>@Column</code>                                                                                  | Customizes column mapping                          | <code>@Column(name = "user_name")</code>                         |
| <code>@OneToMany</code> , <code>@ManyToOne</code> , <code>@OneToOne</code> , <code>@ManyToMany</code> | Define relationships between entities              | <code>@OneToMany(mappedBy = "user")</code>                       |
| <code>@JoinColumn</code>                                                                              | Defines foreign key column                         | <code>@JoinColumn(name = "user_id")</code>                       |
| <code>@Transactional</code>                                                                           | Ensures methods run within a transaction           | <code>@Transactional public void saveUser() { ... }</code>       |

## Spring Boot-Specific Annotations

| Annotation                          | Purpose                                                                                                        | Example                                                   |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>@SpringBootApplication</code> | Combines <code>@Configuration</code> , <code>@EnableAutoConfiguration</code> , and <code>@ComponentScan</code> | <code>@SpringBootApplication public class MyApp {}</code> |

| Annotation                                                          | Purpose                                               | Example                                                                                |
|---------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>@EnableAutoConfiguration</code>                               | Enables automatic configuration based on dependencies | (Included in <code>@SpringBootApplication</code> )                                     |
| <code>@ComponentScan</code>                                         | Tells Spring where to look for components             | <code>@ComponentScan("com.example")</code>                                             |
| <code>@RestControllerAdvice</code> / <code>@ControllerAdvice</code> | Handles exceptions globally                           | <code>@RestControllerAdvice</code> public class <code>GlobalExceptionHandler</code> {} |
| <code>@ExceptionHandler</code>                                      | Handles specific exceptions                           | <code>@ExceptionHandler(Exception.class)</code>                                        |
| <code>@Profile</code>                                               | Activates beans only for a specific environment       | <code>@Profile("dev")</code>                                                           |
| <code>@ConditionalOnProperty</code>                                 | Loads bean based on property value                    | <code>@ConditionalOnProperty(name="feature.enabled", havingValue="true")</code>        |



## Testing Annotations

| Annotation                   | Purpose                                                | Example                                                                           |
|------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>@SpringBootTest</code> | Loads full application context for integration testing | <code>@SpringBootTest</code>                                                      |
| <code>@WebMvcTest</code>     | Loads only web layer for controller tests              | <code>@WebMvcTest(UserController.class)</code>                                    |
| <code>@DataJpaTest</code>    | Loads JPA components for repository tests              | <code>@DataJpaTest</code>                                                         |
| <code>@MockBean</code>       | Creates mock bean for testing                          | <code>@MockBean</code> private <code>UserService</code> <code>userService;</code> |

## 7 Lombok (Optional but Common in Spring Projects)

| Annotation                               | Purpose                                                            | Example                                          |
|------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------|
| @Getter , @Setter                        | Auto-generates getters/setters                                     | @Getter @Setter private String name;             |
| @Data                                    | Generates getters, setters, toString() , equals() , and hashCode() | @Data public class User {}                       |
| @NoArgsConstructor , @AllArgsConstructor | Generates constructors                                             | @AllArgsConstructor                              |
| @Builder                                 | Provides builder pattern for object creation                       | User user =<br>User.builder().name("John").build |